

CUSTOM VPN

Project Completion Report

Mentees :

1. Amit Phadke.
2. Advait Wadke.
3. Shivam Bhalerao.

Mentors :

1. Jinmay Jhaveri.
2. Mudit Bengani.
3. Pratyush Rao.

Contents :

- 1.Introduction
- 2.Technologies used
- 3.Project Architecture
- 4.Implementation / Working
- 5.Limitations
- 6.Unique Feature
- 7.Glimpses of Custom VPN

1.Introduction :

1.1 Overview

Data sent over public networks passes through many devices such as routers, Wi-Fi hotspots and internet providers, and unprotected traffic can be read or modified along the way. A Virtual Private Network (VPN) solves this by creating an encrypted tunnel between a user's device and a trusted server, so the traffic stays private and appears to come from the server.

1.2 Project Description

This project is a custom VPN built from scratch in C++ under the guidance of ProjectX. It consists of a Linux client and a cloud-hosted server that exchange encrypted traffic over UDP, using a virtual network interface (TUN) to capture packets and modern cryptography (Libsodium) to protect them. The aim was to understand and implement each stage of a VPN ourselves instead of relying on ready-made solutions like OpenVPN or WireGuard.

1.3 Objectives

- Build a client-server VPN that tunnels a device's traffic securely.
- Establish a fresh encryption key for every session.
- Protect every packet with authenticated encryption and reject replayed packets.

-
- Allow tunnelled traffic to reach the internet and return to the right client.
 - Provide a simple frontend to run the VPN.

1.4 Scope

The scope of this project is to design and implement a working VPN prototype for Linux, covering the complete path of a packet from the user's application to the internet and back. The project includes:

- A client that captures the device's traffic through a virtual TUN interface and sends it to the server UDP socket.
- A cloud-hosted server that receives client traffic, forwards it to the internet using NAT forwarding rules, and returns the replies.
- Secure session establishment using ephemeral key exchange, with a fresh key for every connection.
- Encryption and integrity protection for every packet, along with basic replay protection.
- Automatic assignment of a virtual IP address to each connected client.

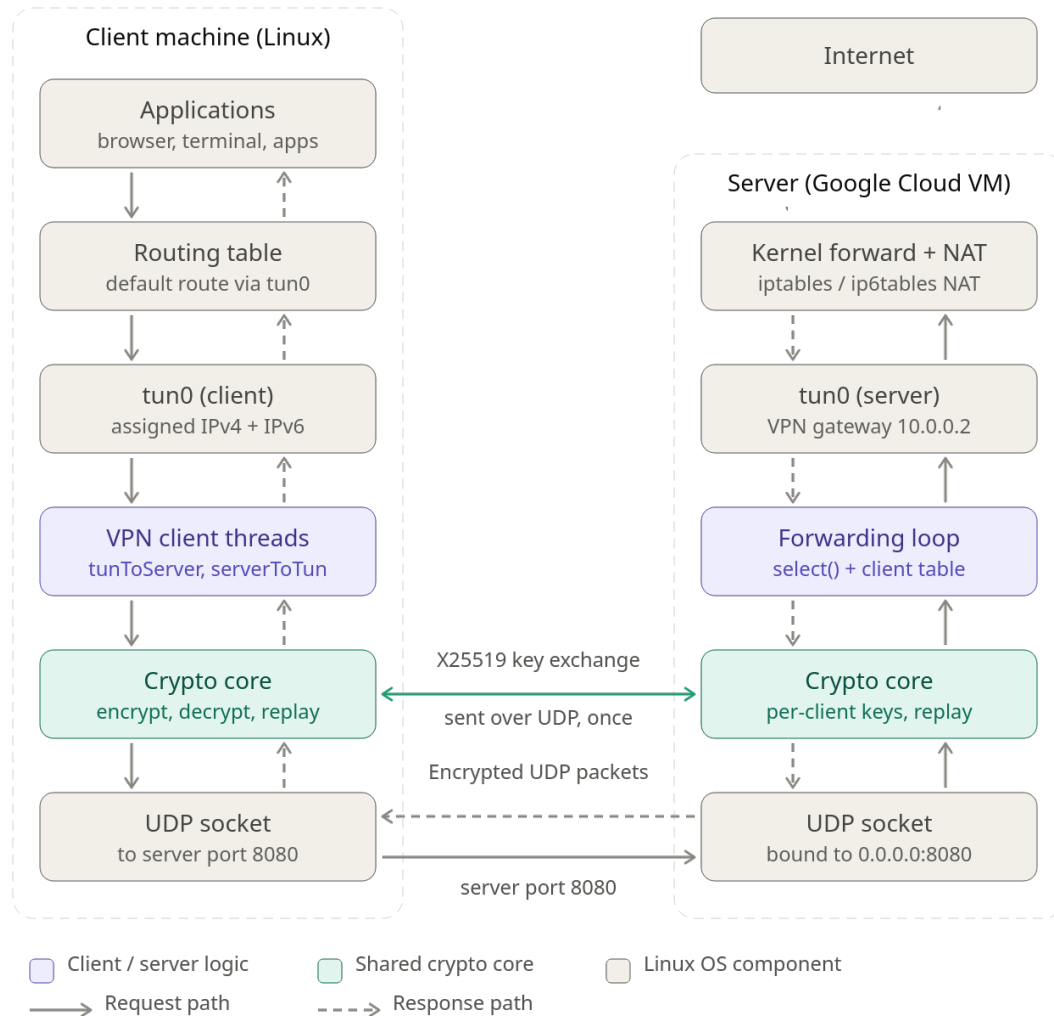
2. Technologies Used :

1. **C++:** The main programming language used to write the client, server and security code.
2. **Linux TUN interface:** A virtual network card (tun0) that lets our program catch the data leaving the computer and put received data back into it.

-
3. **UDP sockets:** A fast way of sending data over the network. It carries our encrypted packets between the client and the server.
 4. **Libsodium:** A trusted security library that gives us ready-made, tested encryption tools, so we don't have to invent our own.
 5. **X25519 key exchange:** A method that lets the client and server agree on a secret key without ever sending that key over the network.
 6. **Key derivation (Libsodium KDF):** Turns that one secret into two separate keys, one for each direction of traffic, for extra safety.
 7. **XChaCha20-Poly1305 encryption:** Scrambles every packet so nobody can read it, and adds a tag that lets the receiver detect if it was changed.
 8. **Sequence numbers and replay window:** Every packet gets a number, and the receiver rejects any packet that is repeated or too old, which stops attackers from resending captured packets.
 9. **Multithreading:** Lets the client send data, receive data and listen for the disconnect command all at the same time.
 10. **iptables and ip6tables:** Linux firewall tools. On the server they let VPN traffic go out to the internet and let the replies come back.
 11. **iproute2 (ip command) and sysctl:** Used to give the VPN its address, set the routes that send traffic through the tunnel, and turn on packet forwarding.

-
12. **IPv4 and IPv6 support:** Each client receives both a private IPv4 and an IPv6 address, and IPv6 traffic is carried inside the IPv4 tunnel.
 13. **Google Cloud Platform (GCP):** Hosts our VPN servers on a cloud machine that has a public IP address.
 14. **Qt 6:** A toolkit used to build the graphical frontend, so the VPN can be used without typing commands.
 15. **CMake:** The build tool that compiles all the code and produces the client, server and frontend programs.
 16. **tcpdump and Wireshark:** Packet-capture and analysis tools that show the packets on the network, used to test and debug the tunnel and to confirm the traffic is encrypted.

3.Project Architecture :



Explanation :

Client-Side Architecture (Linux Machine)

- **Applications and Routing:** Standard applications (like browsers or terminal utilities) generate network traffic, which the host's

routing table automatically directs toward the virtual tun0 interface as the default route.

- **Client TUN Interface (tun0):** This virtual network interface captures the outbound IP packets from the operating system and is assigned dynamically allocated IPv4 and IPv6 addresses by the VPN server.
- **VPN Client Threads:** Dedicated execution threads (tunToServer and serverToTun) extract the raw traffic from the TUN interface to process it, and conversely write incoming internet responses back into the TUN interface.
- **Client Crypto Core:** This component secures the data using the Libsodium library. It encrypts outbound packets using per-session keys via XChaCha20-Poly1305 AEAD, manages decryption, and actively applies replay protection to discard any duplicated or reordered packets.
- **UDP Transport Socket:** The encrypted payload is transmitted out to the internet through a UDP socket targeting the server's IP address on port 8080.

The Secure Tunnel

- **Key Exchange:** The connection strictly begins with a one-time ephemeral X25519 key exchange over UDP. This allows both the client and server to independently calculate a shared secret locally without ever transmitting the secret itself.

-
- **Encrypted Flow:** Once the session keys are successfully derived from the shared secret, all subsequent application traffic travels between the client and server as encrypted UDP packets.

Server-Side Architecture

- **Listening Socket:** The server application binds a UDP socket to 0.0.0.0:8080, allowing it to intercept incoming encrypted packets from multiple remote clients.
- **Server Crypto Core:** The server matches incoming UDP packets to specific per-client cryptographic keys. It decrypts the traffic, validates the AEAD authentication tags, and enforces its own replay protection before accepting the data.
- **Forwarding Loop:** A multiplexed event loop using the `select()` system call references a client table to manage active connections and routes the successfully decrypted IP packets toward the server's virtual interface.
- **Server TUN Gateway:** The server's interface receives the decrypted plaintext packets and injects them directly into the Google Cloud VM's operating system network stack.
- **Kernel Forwarding and NAT:** Because the VPN utilizes private, unroutable IP addresses, the Linux kernel relies on iptables and ip6tables Network Address Translation (NAT) rules to masquerade the traffic. This translates the internal VPN IP to the server's public IP address before forwarding the request to the external Internet.

-
- **Return Path:** When the target website or internet host responds, the traffic follows this exact architectural path in reverse to safely reach the client application.

4.Implementation / Working :

1. TUN Creation

- The TUN interface setup logic is modularized within tun/setup.cpp and linked as part of the vpn_core_objects library.
- It provides a virtual network adapter at the operating system level, which intercepts outgoing internet traffic and allows the VPN client to capture raw IP packets before transmitting them.

2. Client Side

- The client application is compiled into the vpn_client executable.
- It integrates client1.cpp for the main execution loop, transport.cpp for managing the UDP packet exchange, and disconnect.cpp to handle secure session termination.

3. Server Side

- The server application is compiled into the vpn_server1 executable, combining the initialization logic in server.cpp with the routing loop in forward.cpp.

-
- The VPN infrastructure relies on three Google Cloud (GC) servers hosted regionally in the USA (35.226.148.101), Europe/London (34.105.188.210), and Asia/Tokyo (34.84.46.243) to distribute load and optimize latency.

4. Encryption

- Cryptographic operations are powered by the external libsodium library.
- The security model is broken down into dedicated source files for distinct tasks: `handshake.cpp` for initial key exchange, `session_keys.cpp` for deriving symmetric keys, `packet_crypto.cpp` for payload encryption, and `replay_protection.cpp` to validate packet sequences.

5. UI (User Interface)

- The graphical frontend (`vpn_frontend`) is built using the Qt6 framework, specifically relying on the Qt Widgets and Qt Network modules.
- It acts as a control center that executes and communicates with the underlying `vpn_client` binary asynchronously via `QProcess`.
- A live dashboard continuously displays network telemetry, calculating internet speed in KiB/s by reading `/proc/net/dev` and monitoring the host's active local IPv4 address.

-
- An interactive map widget visualizes connection routing by querying the ipapi.co API for IP geolocation data and rendering geographic map tiles retrieved from OpenStreetMap.

5.Limitations :

- the handshake is currently unauthenticated, so the system does not yet protect against active man-in-the-middle attacks
- key rotation is not yet implemented
- packet sequencing and replay protection are still being matured as part of the ongoing protocol design
- routing and network setup are Linux-specific and depend on TUN and iptables/ip6tables tooling.

6.Unique Feature :

Tor Integration Feature (Tor Mode) :

1. Accessing .onion (Hidden) Services

Standard VPNs only route regular internet traffic (clearnet) and cannot resolve or open .onion domain addresses. Tor Mode bridges this gap inside the VPN tunnel, allowing users to seamlessly access hidden services, decentralized platforms, and censorship-resistant sites using the Tor Browser or SOCKS5 applications.

2. Hiding Tor Usage from ISPs and Local Network Monitors

Connecting directly to Tor generates recognizable network signatures that can be detected by ISPs, university networks, or workplace firewalls. By wrapping Tor traffic inside CustomVPN's encrypted UDP tunnel, network administrators only see normal VPN traffic, completely masking the fact that Tor is being used.

3. Masking Real IP Addresses from Tor Entry Nodes

In a standard Tor connection, the first Tor node (the Entry Guard) sees the user's real IP address. With Tor Mode enabled, traffic enters the Tor network directly from one of the Google Cloud VPN servers (USA, London, or Tokyo). The Tor Entry Guard only sees the VPN server's IP address, keeping the user's true IP and geographic location hidden even from the Tor network itself.

4. Bypassing Tor Censorship and Network Blocks

In regions or networks where direct access to Tor entry nodes and directory servers is actively blocked, users cannot establish a standard Tor connection. CustomVPN serves as an encrypted transit layer that bypasses local network filters, allowing users to connect to the Tor network from restricted environments.

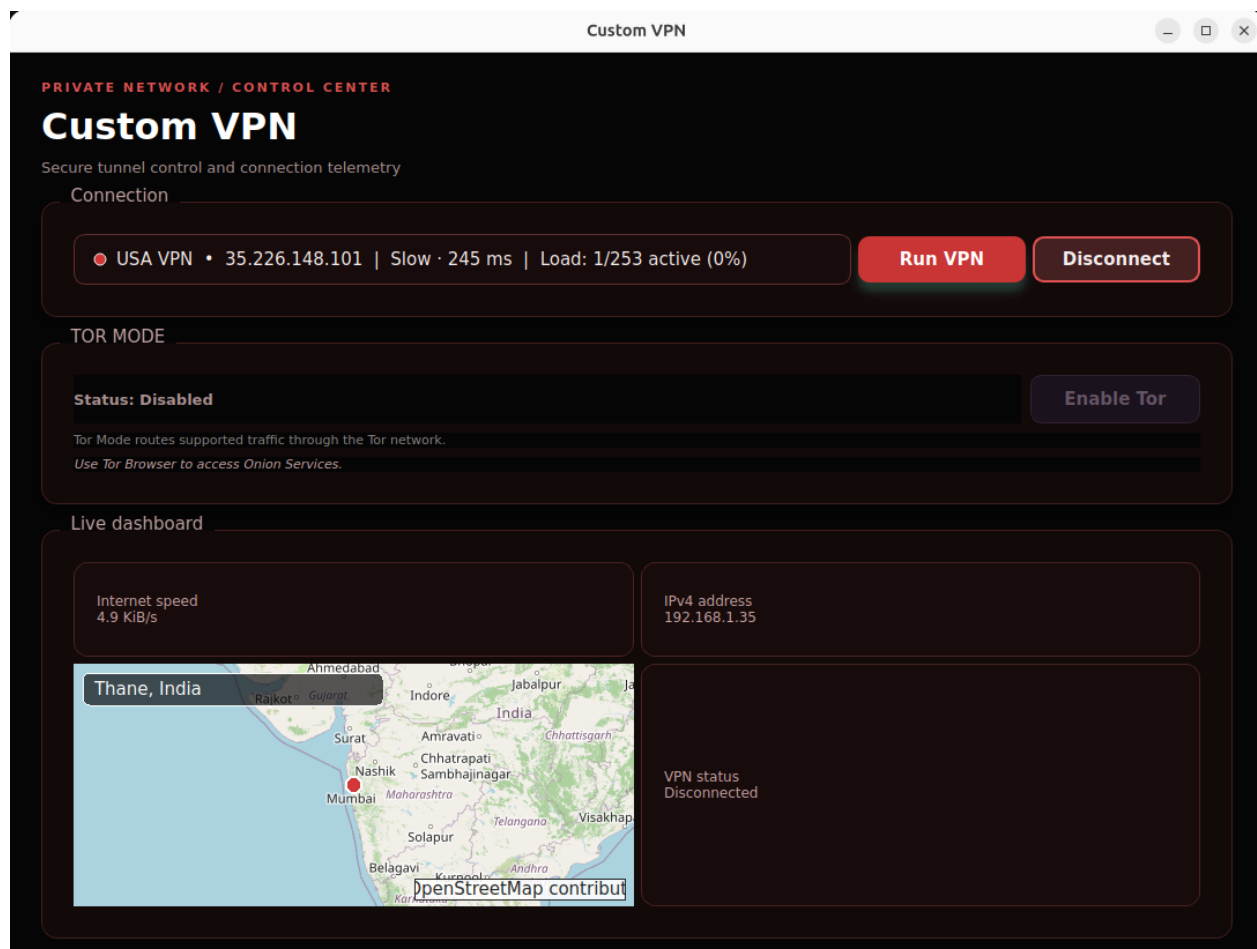
5. Dual-Layer Encryption and Defense-in-Depth Privacy

Tor Mode combines two distinct layers of security: the high-speed, authenticated tunnel of CustomVPN and the multi-hop onion routing of

the Tor network. This setup provides strong defense-in-depth protection for sensitive browsing, privacy research, and whistleblowing activities.

6.Glimpses of Custom VPN :

Before running VPN :



Connected VPN through USA server :

